AMS2 XML Backup Tool — Developer Tutorial

This guide covers everything you need to know to build, edit, and maintain the AMS2 XML Backup Tool.

Table of Contents

Project Overview

The AMS2 XML Backup Tool is a Python GUI app built with tkinter that safely backs up and restores Automobilista 2 Custom AI Driver files.

Key features:

- Zero-deletion guarantee (restore only overwrites + adds, never removes)
- MD5 hash verification for backup integrity
- Preview before restore
- Supports .xml, .xml.orig, .xml.backup, .backup, .txt files
- User-chosen config location
- Custom icon support (.ico for window, .png for header)

Folder Structure

Your project lives here:

```
D:\Coding Projects\AMS2 XML Backup\
```

Files in the folder:

```
D:\Coding Projects\AMS2 XML Backup\
  AMS2_XML_Backup_Tool.py          ← Main Python app (EDIT THIS)
  icon.ico                          ← Window icon (top-left corner)
  AMS2 XML Backup Icon.png          ← Header image (next to title text)
  run_tool.bat                      ← Python launcher (for testing)
```

```
  BUILD_INSTRUCTIONS.md             ← User-facing build guide
  DEVELOPER_TUTORIAL.md             ← This file
```

After building the .exe:

```
D:\Coding Projects\AMS2 XML Backup\
  AMS2 XML Backup Tool.exe          ← Your finished app (from dist/)
  icon.ico                          ← Keep next to .exe (if not using --add-data)
  AMS2 XML Backup Icon.png          ← Keep next to .exe (if not using --add-data)
  build\                            ← Delete after building
  dist\                             ← Delete after building
  AMS2 XML Backup Tool.spec         ← Delete after building
```

Building the .exe

Prerequisites

Make sure these are installed (you already have them):

```
pip install Pillow
pip install pyinstaller
```

Step 1: Navigate to your folder

```
cd "D:\Coding Projects\AMS2 XML Backup"
```

Important: Use quotes because the folder name has spaces.

Step 2: Build the .exe

Standard build (icons as separate files next to .exe):

```
python -m PyInstaller --onefile --windowed --name "AMS2 XML Backup Tool" --icon=icon.ico
↪  AMS2_XML_Backup_Tool.py
```

Self-contained build (icons BUNDLED inside .exe for GitHub):

```
python -m PyInstaller --onefile --windowed --name "AMS2 XML Backup Tool" --icon=icon.ico --add-data
↪  "icon.ico;." --add-data "AMS2 XML Backup Icon.png;." AMS2_XML_Backup_Tool.py
```

Use the self-contained build for GitHub releases. Users download one file and it just works.

Step 3: Wait for build

Takes 30-60 seconds. When you see:

```
Building EXE from EXE-00.toc completed successfully.
```

Step 4: Move your .exe

The .exe is created in `dist\`. Move it to your main folder:

```
D:\Coding Projects\AMS2 XML Backup\dist\AMS2 XML Backup Tool.exe
→ Move to →
D:\Coding Projects\AMS2 XML Backup\AMS2 XML Backup Tool.exe
```

Step 5: Clean up

Delete these (they are only needed during build):

- `build\` folder
- `dist\` folder
- `AMS2 XML Backup Tool.spec` file

Keep these:

- `AMS2 XML Backup Tool.exe`
- `icon.ico` (if not using –add-data)
- `AMS2 XML Backup Icon.png` (if not using –add-data)
- `AMS2_XML_Backup_Tool.py` (source code, for future edits)

Editing the Version Number

The version number appears in the bottom-right corner of the app (e.g., "v1.0").

Using VS Code:

1. **Open VS Code**
   - File → Open Folder → Select `D:\Coding Projects\AMS2 XML Backup`
2. **Open the Python file**
   - Click `AMS2_XML_Backup_Tool.py` in the Explorer sidebar
3. **Find the version**
   - Press **Ctrl+F** (or **Cmd+F** on Mac)
   - Type: `v1.0`
   - VS Code highlights the line:

     ```
     tk.Label(footer, text="v1.0", bg=BG_DARK, fg=TEXT_SECONDARY,
     ```
4. **Edit it**
   - Click on `v1.0` in the code
   - Change to `v1.1`, `v2.0`, etc.
   - Save: **Ctrl+S**
5. **Rebuild the .exe**

```
cd "D:\Coding Projects\AMS2 XML Backup"
python -m PyInstaller --onefile --windowed --name "AMS2 XML Backup Tool" --icon=icon.ico
 ↪  --add-data "icon.ico;." --add-data "AMS2 XML Backup Icon.png;." AMS2_XML_Backup_Tool.py
```

Pro tip: Find & Replace

Use **Ctrl+H** (Find and Replace) to change multiple instances at once:

- **Find:** `v1.0`
- **Replace:** `v1.1`
- Click **Replace All**

Changing Colors

The app uses theme colors defined at the top of the Python file. To change the accent color (currently red `#dc0000`):

1. **Open** `AMS2_XML_Backup_Tool.py` in VS Code
2. **Find the color section** (near the top, around line 30-40):

```
ACCENT = "#dc0000"
ACCENT_DARK = "#a80000"
ACCENT_HOVER = "#f02020"
```

3. **Change the hex values** to any color you want:

```
ACCENT = "#4CC476"        # Green
ACCENT_DARK = "#38B062"   # Dark green
ACCENT_HOVER = "#5AD486"  # Light green
```

4. **Save and rebuild** the .exe

Use a color picker like colorpicker.me to find hex codes.

Adding/Removing File Types

The app backs up these file types by default:

- `.xml`
- `.xml.orig`
- `.xml.backup`
- `.backup`
- `.txt`

To add a new file type:

1. **Open** `AMS2_XML_Backup_Tool.py` in VS Code
2. **Find the scan function** (search for `def scan_xml_files`):

```
patterns = ["*.xml", "*.xml.orig", "*.xml.backup", "*.backup", "*.txt"]
```

3. **Add your new pattern** to the list:

```
patterns = ["*.xml", "*.xml.orig", "*.xml.backup", "*.backup", "*.txt", "*.json"]
```

4. **Save and rebuild** the .exe

To remove a file type:

Delete the pattern from the list:

```
patterns = ["*.xml", "*.xml.orig", "*.xml.backup"]  # Removed .backup and .txt
```

Icon Files

Two icon files are used:

File	Purpose	Required?
`icon.ico`	Windows .exe icon (Explorer, Taskbar, Alt+Tab)	**Yes**
`icon.ico`	Window corner icon (top-left of app window)	**Yes**
`AMS2 XML Backup Icon.png`	In-app header image (next to title)	No

How icons work in different build modes:

Build Type	`icon.ico` in Explorer/Taskbar	Window Corner Icon	Header PNG
Standard (no –add-data)	✅ Embedded by `--icon`	✅ Loaded from .exe folder	✅ Loaded from .exe folder
Self-contained (–add-data)	✅ Embedded by `--icon`	✅ Loaded from `sys._MEIPASS` temp folder	✅ Loaded from `sys._MEIPASS` temp folder

Converting PNG to ICO:

If you have a PNG and need an ICO:

1. Go to cloudconvert.com or convertio.co
2. Upload your PNG
3. Download the ICO file
4. Name it `icon.ico`

Recommended sizes:

- **ICO:** 256x256 pixels (Windows scales it down automatically)
- **PNG header:** Any size (app scales to 48x48 automatically)

Common Issues & Fixes

Issue: "pyinstaller is not recognized"

Fix: Use Python's module syntax:

```
python -m PyInstaller --onefile --windowed --name "AMS2 XML Backup Tool" --icon=icon.ico
↪  AMS2_XML_Backup_Tool.py
```

Issue: SyntaxError when running the app

Cause: Multi-line strings broken during editing (newlines inside quotes).

Fix: Make sure string literals don't have actual line breaks inside them. Use `\n` for newlines within strings.

Issue: Icons not showing in the .exe

Cause: PyInstaller onefile extracts bundled files to a **temporary folder** (`sys._MEIPASS`), not the .exe folder. The code was looking for icons in the wrong place.

Fix: The code now uses `_get_app_dir()` which checks `sys._MEIPASS` when running as .exe:

```
def _get_app_dir(self) -> Path:
    if hasattr(sys, '_MEIPASS'):
        # PyInstaller onefile: files extracted to temp folder
        return Path(sys._MEIPASS)
    else:
        # Python script: files in .py folder
        return Path(__file__).parent
```

Make sure:

1. You used `--add-data "icon.ico;."` and `--add-data "AMS2 XML Backup Icon.png;."` when building
2. The icon files are in the same folder as your `.py` file before building
3. Rebuild the .exe after any icon changes

Issue: App window doesn't appear

Cause: The first-time setup dialog might be hidden behind another window.

Fix: The code now shows the main window first, then the dialog on top. If it still hangs, delete the config file and restart.

Issue: Config file location wrong

Fix: Click **SETTINGS** in the app, or delete the config file:

- Home: `C:\Users\[You]\.ams2_xml_backup_config.json`
- Documents: `Documents\AMS2_XML_Backup_Tool\.ams2_xml_backup_config.json`
- Custom: wherever you chose

The app will ask again on next launch.

Full Build Command Reference

For testing (Python):

```
cd "D:\Coding Projects\AMS2 XML Backup"
python AMS2_XML_Backup_Tool.py
```

Or double-click `run_tool.bat`.

For personal use (icons as separate files):

```
cd "D:\Coding Projects\AMS2 XML Backup"
python -m PyInstaller --onefile --windowed --name "AMS2 XML Backup Tool" --icon=icon.ico
↪  AMS2_XML_Backup_Tool.py
```

For GitHub distribution (self-contained .exe):

```
cd "D:\Coding Projects\AMS2 XML Backup"
python -m PyInstaller --onefile --windowed --name "AMS2 XML Backup Tool" --icon=icon.ico --add-data
↪  "icon.ico;." --add-data "AMS2 XML Backup Icon.png;." AMS2_XML_Backup_Tool.py
```

Clean build (delete old files first):

```
cd "D:\Coding Projects\AMS2 XML Backup"
rd /s /q build
rd /s /q dist
del "AMS2 XML Backup Tool.spec"
python -m PyInstaller --onefile --windowed --name "AMS2 XML Backup Tool" --icon=icon.ico --add-data
↪  "icon.ico;." --add-data "AMS2 XML Backup Icon.png;." AMS2_XML_Backup_Tool.py
```

Distribution (GitHub)

For GitHub Releases, use the self-contained build:

```
python -m PyInstaller --onefile --windowed --name "AMS2 XML Backup Tool" --icon=icon.ico --add-data
↪  "icon.ico;." --add-data "AMS2 XML Backup Icon.png;." AMS2_XML_Backup_Tool.py
```

Upload to GitHub:

1. Go to your GitHub repo → Releases → Draft a new release
2. Upload `AMS2 XML Backup Tool.exe` (just one file)
3. Users download and run it — no extra files needed

Why this works:

- `--add-data "icon.ico;."` bundles the .ico INTO the .exe
- `--add-data "AMS2 XML Backup Icon.png;."` bundles the PNG INTO the .exe
- At runtime, PyInstaller extracts them to a temp folder (`sys._MEIPASS`)
- The `_get_app_dir()` function finds them automatically via `sys._MEIPASS`

How __get__app__dir() Works

This is the **critical function** that makes icons work in both `.py` script mode and `.exe` bundled mode.

The Problem

PyInstaller's `--onefile` mode works like this:

1. Your `.exe` is a self-extracting archive
2. When launched, it creates a **temporary folder** named `_MEIxxxxxx`
3. All bundled files (including your `--add-data` icons) are extracted there
4. Your Python code runs from that temp folder
5. When the app closes, the temp folder is deleted

So `Path(__file__).parent` points to the temp folder, **not** your `.exe` folder. And `Path(sys.executable).parent` points to your `.exe` folder, **not** the temp folder where the icons were extracted.

The Solution

PyInstaller sets a special attribute `sys._MEIPASS` that contains the path to the temp extraction folder. We check for this attribute:

```
def _get_app_dir(self) -> Path:
    if hasattr(sys, '_MEIPASS'):
        # PyInstaller onefile: files extracted to temp folder
        return Path(sys._MEIPASS)
    else:
        # Python script: files in .py folder
        return Path(__file__).parent
```

How it behaves:

Mode	`hasattr(sys, '_MEIPASS')`	Returns	Where icons are found
`.py` script	`False`	`Path(__file__).parent`	Your `AMS2 XML Backup` folder
`.exe` with `--add-data`	`True`	`Path(sys._MEIPASS)`	PyInstaller's temp `_MEIxxxxxx` folder

What this means for you:

- **When testing as .py:** Icons load from your project folder

- **When running as .exe:** Icons load from the bundled temp folder
- **No code changes needed** between modes — it auto-detects

Quick Reference Card

Task	Command/Action
Test run	Double-click `run_tool.bat`
Build .exe	`python -m PyInstaller --onefile --windowed --name "AMS2 XML Backup Tool" --icon=icon.ico --add-data "icon.ico;." --add-data "AMS2 XML Backup Icon.png;." AMS2_XML_Backup_Tool.py`
Change version	Edit `v1.0` in `AMS2_XML_Backup_Tool.py`, save, rebuild
Change color	Edit `ACCENT = "#dc0000"` hex value, save, rebuild
Add file type	Add `"*.ext"` to `patterns` list in `scan_xml_files()`, save, rebuild
Reset config	Delete `.ams2_xml_backup_config.json`, restart app
Clean build	Delete `build\`, `dist\`, `.spec`, then rebuild

Need Help?

If something breaks:

1. Check the Status Log in the app
2. Run from Command Prompt to see full error output: `bash cd "D:\Coding Projects\AMS2 XML Backup" python AMS2_XML_Backup_Tool.py`
3. Copy the error message and search online or ask for help